

Modul 14 Scripting

Tujuan Praktikum

1. Praktikan dapat melakukan otomasi dalam OS.
2. Praktikan dapat melakukan scripting OS.

14.1 Shell

14.1.1 Bash

Bash (Bourne-Again Shell) adalah unix shell (command line interpreter/command line user interface) yang ditulis oleh Brian Fox sebagai pengganti dari Bourne shell. Bash adalah command processor yang biasanya berjalan mode text dimana user mengetikkan perintah di layar untuk kemudian dieksekusi oleh komputer. Bash dapat juga membaca dan mengeksekusi perintah dari file yang disebut *bash script*.

```
kar@karshain: /usr/portage/app-shells/bash $ grep -i /dev/sda /etc/fstab | cut --fields=3
/dev/sda1      /boot
/dev/sda2      none
/dev/sda3      /
kar@karshain: /usr/portage/app-shells/bash $ date
Sat Aug  8 02:42:24 MSD 2009
kar@karshain: /usr/portage/app-shells/bash $ lsmod
Module                  Size  Used by
rndis_wlan              23424  0
rndis_host              8696   1 rndis_wlan
cdc_ether               5672   1 rndis_host
usbnet                 18688   3 rndis_wlan,rndis_host,cdc_ether
parport_pc             38424   0
lpr                    2388128  20
parport                39648   1 parport_pc
ltdc_udt               12272   0
ltdc_i801              9380    0
kar@karshain: /usr/portage/app-shells/bash $
```

Gambar 14-1 Contoh Sesi Bash

Bash script adalah *plain text* yang berisi urutan perintah-perintah. Dari pada menulis satu per satu perintah ke dalam terminal, dengan adanya *script*, perintah akan dieksekusi sesuai dengan urutan yang ada dalam *script*. Pada umumnya *bash script* diberi ekstensi *.sh* (walaupun bukan sebuah keharusan).

Menjalankan *bash script* sangat mudah yaitu hanya dengan mengeksekusi *script* tersebut. Jangan lupa untuk mengubah *permission* dari *script* jika tidak bisa dijalankan (dengan memberikan perintah `chmod +x myscript.sh` atau `chmod 755 myscript.sh`).

```
$ ./myscript.sh
Hello World!
```

14.1.2 Struktur Bash

Berikut adalah contoh sederhana dari *bash script*:

```
myscript.sh
1. #!/bin/bash
2. # A sample Bash script
3.
4. echo Hello World!
```

Gambar 14-2 myscript.sh

Mari kita analisis baris per baris:

- Baris 1: disebut sebagai **shebang**. Karakter **#!** Dikenal sebagai **shebang** diikuti dengan *path* ke interpreter (dalam hal ini adalah *bash*). Letak program *bash* ada di */bin/bash*. Format sangat penting. **Shebang** harus berada pada baris pertama (jika diletakkan pada baris ke dua tidak akan berjalan). Tidak boleh ada spasi antara **#** dan **!** dan path ke interpreter.
- Baris 2: contoh komentar, apapun perintah setelah **#** tidak akan dieksekusi.
- Baris 3: *spacing* agar mempermudah pembacaan.
- Baris 4: perintah yang akan dieksekusi. Pada contoh ini perintah *echo* (menampilkan pesan ke terminal) akan dieksekusi.

14.2 Scripting Dasar

14.2.1 Variabel

Variabel adalah tempat menyimpan informasi sementara. Terdapat 2 hal yang dapat dilakukan pada variabel yaitu *setting* dan *getting* variabel.

Setting Variabel

Pola dasar setting variabel adalah nama_variabel=value. Perhatikan bahwa tidak ada spasi setelah tanda **=**.

- *Single quote* akan memperlakukan setiap karakter secara literal (sesuai dengan yang ditulis).
- *Double quote* memungkinkan adanya substitusi variabel

```
user@bash: myvar='Hello World'
user@bash: echo $myvar
Hello World
user@bash: newvar="More $myvar"
user@bash: echo $newvar
More Hello World
user@bash: newvar1='More $myvar'
user@bash: echo $newvar1
More $myvar
```

Getting Variabel

Untuk membaca variabel gunakan tanda **\$** pada variabel yang ingin diakses.

```
#!/bin/bash
myvar=hello
var1=world
echo $myvar $var1
```

Command Substitution

Command substitution memungkinkan kita untuk menulis suatu perintah dan menyimpan perintah tersebut ke dalam variabel. Untuk melakukan hal tersebut, perintah harus berada dalam **()** dan diawali dengan tanda **\$**.

```
user@bash: myvar=$(ls /etc)
user@bash: echo Terdapat file $myvar di folder /etc
```

14.2.2 Command Line Argument

Command line argument adalah argumen yang diberikan pada perintah/*script*.

Contoh: `myscript.sh /home/data.zip /tmp`

Pada contoh tersebut terdapat 2 *command line argument* yaitu: `/home/data.zip` dan `/tmp`. `/home/data.zip` disimpan dalam variabel `$1` sedangkan `/tmp` disimpan dalam variabel `$2`.

```
user@bash: myscript.sh /home/data.zip /tmp
user@bash: echo $1
/home/data.zip
user@bash: echo $2
/tmp
user@bash: cp $1 $2
```

variabel spesial lainnya:

- `$0` = *nama script*
- `$1-$9` = argumen 1 s.d 9
- `$$` = *pid script*
- `$USER` = nama *user* yang menjalankan *script*
- `$RANDOM` = angka *random*

14.2.3 Input

Untuk meminta *input* dari *user* dapat digunakan perintah ***read***.

```
#!/bin/bash
echo Hello, Who are you?
read varname
echo Nice to meet you $varname

echo What cars do you like?
read car1 car2 car3

echo your first car: $car1
echo your second car: $car2
echo your third car: $car3
```

14.2.4 Aritmatik

Bash mempunyai fungsi *built-in* untuk melakukan aritmatik. Terdapat beberapa teknik untuk melakukan aritmatik yaitu: ***let*** dan ***expr***.

Let

Format dasarnya perintah adalah ***let*** <ekpresi aritmatik>.

Bagian pertama biasanya adalah variabel dimana hasil akan disimpan. Berikut adalah contoh menggunakan aritmatik:

```
#!/bin/bash
let a=20+22
echo $a
let "a = 5 + 4"
echo $a
let a++
echo $a
let "a = $1 + 10"
echo $a # 10 + argument pertama
```

Hasil, jika dieksekusi contoh `_let.sh 13`

```
$
42
9
10
```

Operator

$+, -, *, /$ = tambah, kurang, kali, bagi

$Var++$ = *increment*

$Var--$ = *decrement*

$\%$ = modulo

Expr

Expr dapat digunakan substitusi dan tidak perlu menggunakan *quote*. Perlu diperhatikan: harus ada spasi antar *item* dalam ekspresi.

```
#!/bin/bash
expr 5 + 4
expr "5 + 4"
expr 5+4 #jika tidak ada spasi maka tidak dievaluasi, hanya diprint
expr 5 \* $1
a=$(expr 10-3)
echo $a
```

Hasil, jika dieksekusi contoh_expr.sh 6

```
$
9
9
5+4
30
7
```

14.2.5 If Statement

Format *if statement* adalah sebagai berikut:

```
if [some test]
then
    <perintah>
else
    <perintah lain>
fi
```

Contoh:

```
#!/bin/bash
if [$1 -ge 17]
then
    echo Boleh melakukan pemilu
else
    echo Tunggu pemilu selanjutnya
fi
```

If statement dapat *dinested* jika diperlukan.

Test

Terdapat beberapa *test* yang dapat dilakukan dalam *if []* yaitu:

$String1 = String2$ *string* 1 sama dengan *string* 2

$String1 != String2$ *string* 1 tidak sama dengan *string* 2

Integer1 -eq integer 2	integer 1 sama dengan integer 2
Integer1 -gt integer 2	integer 1 lebih besar dari integer 2
Integer1 -lt integer 2	integer 1 lebih kecil dari integer 2
-e file	file ada
&&	logika <i>and</i>
	logika <i>or</i>

Case

```
case <variabel> in
<pola 1>
    perintah
    ;;
<pola 2>
    perintah
    ;;
esac
```

Contoh:

```
#!/bin/bash
case $1 in
start)
    echo Mulai
    ;;
stop)
    echo Berhenti
    ;;
esac
```

14.2.6 Loop

Terdapat 3 dasar iterasi yaitu *for*, *while* dan *until*.

For

```
for var in <list>
do
    perintah
done
```

Contoh:

```
#!/bin/bash
for value in {1..10}
do
    echo $value
done

for value in {10..0..2}
do
    echo $value
done
```

While

```
while [some test]
```

```
do
    perintah
done
```

Contoh:

```
#!/bin/bash
counter=1
while [$counter -le 10]
do
    echo $counter
    ((counter++))
done
```

Until

```
until [some test]
do
    perintah
done
```

Contoh:

```
#!/bin/bash
counter=1
until [$counter -gt 10]
do
    echo $counter
    ((counter++))
done
```

Untuk mengendalikan iterasi dapat digunakan perintah **continue** dan **break**. Perintah ini mirip dengan perintah pada bahasa C.

14.2.7 Function

Fungsi digunakan untuk *code reuse*. Jika terdapat perintah-perintah yang sering digunakan maka dapat dibuat sebuah fungsi sehingga kita hanya akan memanggil fungsi tersebut dan bukan menulis ulang *code* dari awal.

```
nama_fungsi(){
    Perintah-perintah
}
```

Contoh:

```
#!/bin/bash

tampilkan_sesuatu(){
    echo Hello $1
}

tampilkan_sesuatu Mars
tampilkan_sesuatu Jupiter
```

Hasil:

```
$/myfunction.sh
Hello Mars
Hello Jupiter
```